AI CODING WORKSHOP | 체험

말로 만들고,
알고 다듬는 코딩

AI 기반 Vibe Coding · 입문 워크숍



처음에는 만드는 감각을.

다음에는 구조와 보는 안목을.

마지막에는 직접 만드는 경험을.

9월 29일 · 16:00–17:00

누구나 시작할 수 있습니다. 지식과 안목이 더 좋은 결과를 만듭니다.

첫 요청에서 시작합니다

설명보다 먼저, 말이 작동하는 결과로 이어지는 과정을 봅니다.

나의 요청

별을 누르면 점수가 올라가는
간단한 게임을 만들어줘.

제작 요청 보내기

요청을 읽고 파일을 만들고 실행한 과정을
재현합니다.

AI 작업은 준비된 재현입니다.
오른쪽 게임은 브라우저에서 실제로 작동합니다.



첫 결과를 보니, 바꾸고 싶은 것이 생깁니다.

결과를 보고, 한 번 더 요청합니다

"잘 만들어줘"에서 "무엇을 어떻게 바꿀지"로 요청이 구체적이 됩니다.

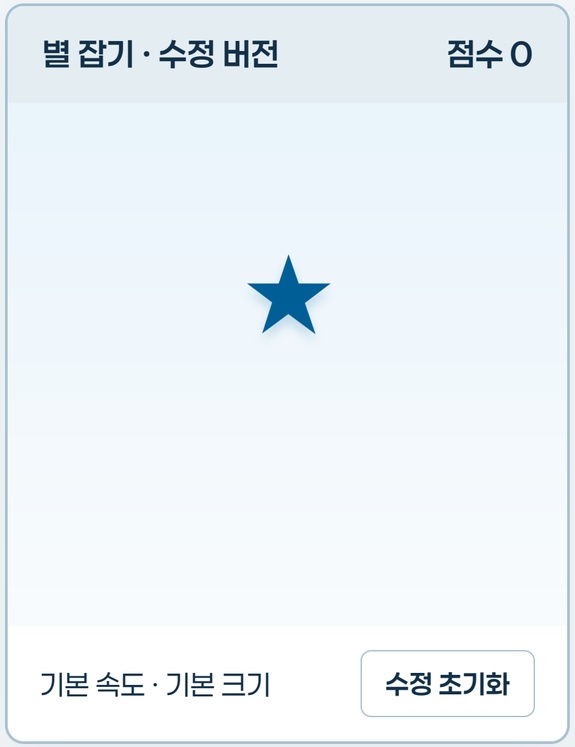
별이 너무 빨라. 천천히 움직여줘.

점수가 눈에 잘 들어오게 해줘.

별을 더 크게 해서 누르기 쉽게 해줘.

> 어떤 점을 먼저 바꾸고 싶으신가요?

세 가지 요청에 대응하는 변화가 준비되어 있습니다.
이 화면에서 실제 AI를 호출하지는 않습니다.



누가 만들었고, 누가 좋은 결과의 기준을 정했을까요?

사람의 판단이 작업의 방향을 정합니다

방금 사람이 한 일은 코드를 쓰는 것보다, 원하는 경험을 설명하는 일이었습니다.

01

목적을 정합니다

"쉽게 즐길 수 있는 게임"
무엇을 만들지 결정합니다.

02

AI가 구현합니다

허용된 도구로 파일을 만들고 수정하며 실행을 돕습니다.

03

결과를 판단합니다

"빠르다", "잘 안 보인다"
사용 경험과 정답을 확인합니다.

요청하는 것도 사람의 일.

무엇을 고칠지 알아보는 것도 사람의 일.

AI의 답변이 실제 파일 작업으로 이어지는 이유도 살펴봅니다.

답변과 실제 작업의 차이

같은 대화창처럼 보여도, 연결된 도구와 작업 공간에 따라 할 수 있는 일이 달라집니다.

설명을 받는 대화

“이렇게 고치세요.”

코드나 방법을 답변으로 받습니다.
파일에 적용하고 실행할 일이 남습니다.

버튼 크기를 80px로 바꾸세요.

코딩 에이전트

“파일을 확인하고 고치겠습니다.”

작업 폴더의 파일과 도구를 사용합니다.
수정 내용과 실행 결과를 확인합니다.

파일 찾기 | 수정하기 | 실행하기

제품 이름보다 **어떤 도구와 권한이 연결되어 있는지**가 중요합니다.

실제로 작업해 주어도, 무엇을 요구하고 받아들일지는 사람이 판단합니다.

지식과 안목이 결과를 바꿉니다

자연어는 전달 방식입니다. 무엇을 요구하고 어떤 결과를 받아들일지는 실력입니다.



알면 더 구체적으로 요구합니다

화면과 처리의 역할을 이해하면, 어디를 왜 바꿀지 설명할 수 있습니다.

보이면 더 나은 결과를 고릅니다

GUI의 정보 배치, 가독성, 조작 흐름을 구별할수록 개선 방향이 분명해집니다.

경험이 쌓이면 문제를 좁힙니다

원하는 결과가 나오지 않을 때 무엇을 확인할지 판단할 수 있습니다.

그래서 기초 개념을 배웁니다. 코드를 외우기보다, 작업을 이해하기 위해서입니다.

작업 공간에는 무엇이 있을까요?

AI가 만든 결과는 대화창 밖의 파일에 남습니다.



프로젝트 폴더는 코드와 자료를 함께 두는 작업 공간입니다.
처음에는 여러 역할을 HTML 파일 하나에 담을 수도 있습니다.

파일을 만들었으니, 이제 실행해서 결과를 봅니다.

파일은 실행해야 결과가 됩니다

어떤 파일을 만들었는지에 따라 실행 방법도 달라집니다.

HTML 게임 | Python 프로그램

브라우저에서 열기

`index.html`

HTML 파일을 브라우저로 열면 화면을 볼 수 있습니다.

실행 과정 보기

설명용 실행 재현입니다. 실제 셸 명령은 실행하지 않습니다.

터미널

명령을 입력하고 결과를 보는 창입니다. 에이전트도 이 창을 통해 개발도구를 실행할 수 있습니다.

Python

Python으로 쓴 프로그램을 실행하려면 Python 실행 환경이 필요합니다.

모르는 실행 방법은 그대로 질문합니다

“이 파일을 실행하려면 무엇이 필요한지 설명하고 실행을 도와줘.”

실행된 화면에도 구조, 모양, 동작이라는 서로 다른 역할이 있습니다.

화면의 구조, 모양, 동작

하나씩 더하면 같은 화면이 어떻게 달라지는지 보입니다.

HTML | + CSS | + JavaScript



JavaScript가 버튼의 입력을 받아 상태를 바꿉니다.

HTML

제목, 입력란, 버튼 등 화면에 무엇이 있는지 정합니다.

CSS

글자 크기, 색, 간격과 배치 등 어떻게 보일지 정합니다.

JavaScript

클릭이나 입력에 반응하고 화면과 데이터를 바꿉니다.

화면에서 처리할 수도 있고, 서버에 일을 요청할 수도 있습니다.

화면 뒤의 처리와 저장

화면은 보여주고, 처리 부분은 기준을 적용하고, 저장소는 결과를 보관합니



프론트엔드

사용자가 보고 조작하는 부분

백엔드

서버에서 요청을 처리하는 부분

데이터베이스

데이터를 저장하고 찾아 쓰는 부분

> 앞에서 본 작은 게임은 브라우저 안에서 실행됩니다.
> 별도의 서버와 데이터베이스가 모든 프로그램에 필요한 것은 아닙니다.

서버가 있는 경우에는 요청과 결과가 어떻게 오갈까요?

한 번의 입력이 지나가는 길

브라우저 안에서 계산 | 서버에 처리 요청

같은 점수 입력도, 처리 위치에 따라 경로가 달라집니다.

입력
점수 85

→

브라우저 계산
80점 이상인가?

→

결과
기준 충족

작은 순위 도구는 JavaScript로 브라우저 안에서 계산할 수 있습니다.

입력 보내기

> API는 프로그램이 정해진 방식으로 요청하고 결과를 받는 접점입니다.
> 서버에 맡기면 연결 상태와 오류 응답도 함께 다뤄야 합니다.

구조를 알았다면, 이제 결과의 품질을 보는 기준을 살펴봅니다.

둘 다 작동합니다. 어느 쪽이 쓰기 편한가요?

좋은 GUI를 보는 안목은 예쁜 색을 고르는 것보다 넓습니다.

- [x] 중요한 정보를 먼저
- [x] 읽기 편한 간격
- [x] 분명한 버튼 이름

같은 정보와 기능을 유지하면서
사용 경험을 바꿔 봅니다.

후보 검토

검토 시작

검토할 후보 3건

기준 점수 80점 · 2건 충족

후보 A	92	충족
후보 B	85	충족
후보 C	74	미달

마지막 변경 09.29

"더 예쁘게"보다 "무엇을 먼저 보고, 어떤 행동을 하게 할지"가 구체적인 기준입니다.

그럴듯한 결과도 틀릴 수 있습니다

화면이 정상이어도, 처리 기준이 빠졌을 수 있습니다.

업무 기준 · 80점 이상이면 충족

검토 점수

80

보통 값 85

경계값 80

입력 없음

잘못된 값

누락·경계 조건까지 수정

프로그램 결과

충족

기대 결과: 충족

이 입력에서는 맞습니다. 다른 조건도 함께 확인합니다.

차이를 보여주기 위해 첫 버전에는 의도적으로 잘못된 비교와 누락 처리를 넣었습니다.

사람이 업무 기준을 알고 있어야, 빠진 조건도 찾아낼 수 있습니다.

같은 “고쳐줘”에도 관찰의 깊이가 다릅니다

알고 보는 사람은 문제를 더 구체적으로 설명할 수 있습니다.

화면을 보는 눈 | 처리를 이해하는 지식 | 구조를 이해하는 지식

막연한 요청

화면을 더
예쁘게 해줘.

관찰이 담긴 요청

검토할 건수를 먼저
보여주고,
주요 버튼의 이름을
‘검토 시작’으로
바꿔줘.

무엇이 불편한지 볼 수 있으면, 원하는 변화도 구체적으로 말할 수 있습니다.

더 잘 만드는 방법을 배우는 이유는, 좋은 요구와 판단을 하기 위해서입니다.

기준과 변경을 다음 작업에도 이어갑니다

대화에서 합의한 기준을 남기고, 확인한 변경을 기록합니다.

AGENTS.md · 작업 기준

화면은 한국어로 작성합니다.

입력이 없으면 확인을 요청합니다.

수정 후 정상·경계·누락 입력을 확인합니다.

지원하는 에이전트가 읽는 지침 파일입니다.
지침을 적어도 준수 여부는 결과로 확인합니다.

Git · 변경 이력

01 첫 화면

02 처리 수정

03 화면 개선

화면 개선

주요 결과를 먼저 보여주고 버튼 이름을 정리했습니다.

버튼은 저장된 상태를 비교하는 재현입니다.
실제 저장소의 이력을 바꾸지는 않습니다.

이제 이런 판단을 실제 제작 과정에 써보겠습니다.

이제 실제 에이전트에 요청합니다

10분 제작 · 후보 이름과 점수를 넣으면 순위를 보여주는 작은 도구

첫 요청

이름과 점수를 한 줄에 하나씩 입력하면 높은 점수순으로 정리하는 도구를 만들어줘. 브라우저에서 바로 여는 HTML 파일 하나로 만들고, 한국어로 보여줘. 먼저 입력과 결과가 보이는 간단한 버전을 만들어줘.

요청문 복사

실제 제작 시간

10:00

시작 초기화

1분 · 목적 | 4분 · 제작 | 2분 · 확인 | 2분 · 수정 | 1분 · 재확인

여기서는 실제 사용 환경으로 전환합니다. 다음 장에는 진행을 이어갈 수 있는 준비본이 있습니다.

첫 결과를 직접 확인합니다

준비된 첫 버전 · 외부 AI 없이 브라우저 안에서 정렬하는 도구

이름, 점수 · 한 줄에 한 명

```
포석호, 85
하늘, 92
바다, 74
```

정상 입력 | 같은 점수 | 점수 누락 | 빈 입력

실제 생성이 지연되면 준비본임을 밝히고 이어갑니다.
입력한 자료는 외부로 전송하지 않습니다.

순위 결과 3명

1	하늘	92
2	포석호	85
3	바다	74

보기 좋게 정렬됐습니다. 같은 점수와 빠진 값은 어떻게 다뤄야 할까요?

수정할 기준을 말하고 다시 확인합니다

실제 에이전트에 수정 요청을 보낸 뒤, 처음 입력과 예외 입력을 함께 확인합니다.

추가 요청

같은 점수는 공동 순위로 표시해줘. 1위가 두 명이면 다음은 3위야. 점수가 비었거나 숫자가 아니거나 0~100 범위를 벗어나면 해당 줄을 알려줘. 80점 이상을 강조하고, 이름과 점수가 먼저 보이게 화면을 정리해줘. 정상·동점·누락 입력을 다시 확인해줘.

수정 요청 복사

정상 | 동점 | 누락 | 범위 밖

정상 · 높은 점수부터

1위	하늘	92점
2위	포석호	85점
3위	바다	74점

오른쪽은 수정 기준을 반영한 준비본입니다.
실제 생성 결과도 같은 입력으로 별도 확인합니다.

AI로 만들었지만, 이 정렬 도구가 실행될 때 AI를 부르는 것은 아닙니다.

AI로 제작하기와 AI 기능 넣기

AI가 코드를 작성했는지와 프로그램이 AI를 호출하는지는 서로 다른 질문입니다.

AI로 만든 프로그램

점수 정렬

85, 92, 74

정해진 코드로 계산

92, 85, 74

실행할 때 AI 호출 없이 동작합니다.
앞에서 만든 도구가 여기에 해당합니다.

실행 중 AI를 사용하는 프로그램

문장 분류

“접속이 자꾸 끊겨요.”

AI 서비스에 분류 요청

접속 문제

AI의 결과를 받아 화면에 활용합니다.
분류가 맞는지 확인할 기준도 필요합니다.

프로그램 안에서 AI 기능을 쓰려면 서비스와 연결해야 합니다.

AI 요청이 지나가는 경로

P-GPT를 연결하는 경우의 개념도입니다. 실제 주소와 지원 형식은 사내 안내로 확인합니다.

분류할 문장

접속이 자꾸 끊겨요.

정상 응답 | 연결 실패

요청 흐름 보기

미리 정한 규칙으로 응답을 재현합니다.
실제 P-GPT 호출이나 AI 분류가 아닙니다.

화면
사용자 입력

↓

백엔드
API 키는 서버 측에서 사용

↓

P-GPT API
아직 요청하지 않았습니다.

연결 방식뿐 아니라, AI가 틀리거나 응답하지 않을 때도 생각해야 합니다.

연결 전에 확인할 내용

개념을 이해한 다음, 실제 사용 환경에 맞는 정보를 확인합니다.

01

사용 가능한 방식

사내 신청 경로와 사용 권한,
호출 주소·요청 형식을
확인합니다.

02

키와 입력 자료

키를 공유 화면이나 코드에
노출하지 않고,
입력 가능한 자료 범위를
확인합니다.

03

결과와 실패 처리

AI 응답이 틀리거나 연결이
끊겼을 때
사용자에게 무엇을 보여줄지
정합니다.

> 환경변수로 실행 설정을 분리할 수 있습니다.
> 환경변수 자체가 키를 암호화하거나 노출을 자동으로 막아주지는 않습니다.

사내 신청 링크·모델·주소·지원 기능은 이 자료에서 확인되지 않았습니다.

무엇을 만들지에 따라 준비가 달라집니다

HTML 도구 | Python 처리 | AI 연결 도구

먼저 사용할 에이전트와 작업 폴더를 정하고, 실행에 필요한 것만 준비합니다.

작업 도구	작업 공간	실행 환경
코딩 에이전트	프로젝트 폴더	브라우저

오늘의 단일 HTML 예제는 별도의 Python·Node.js 설치 없이 브라우저에서 실행됩니다.

> 라이브러리는 이미 만들어진 기능을 가져다 쓰는 방법입니다.
> Python 가상환경은 프로젝트별 패키지를 분리합니다.
> 필요해질 때 배우면 됩니다.

막히면 현재 상황을 보여줍니다

설명을 부탁하는 것과 작업을 부탁하는 것을 함께 사용할 수 있습니다.

뜻이 낯설 때	“지금 나온 API가 이 프로그램에서 무슨 역할인지 설명해줘.”
실행이 안 될 때	“이 오류가 났어. 원인을 확인하고, 필요한 작업을 진행해줘.”
결과가 이상할 때	“80점도 충족이어야 하는데 미달로 나와. 이 기준으로 고쳐줘.”

현재 입력, 실제 결과, 기대한 결과를 함께 보여주면
어디서 차이가 생겼는지 확인하기 쉽습니다.

배운 만큼 보이고, 보이는 만큼 다듬습니다

작은 제작 경험에 구조 이해와 GUI 안목을 계속 더합니다.



하나 만들고

입력과 결과가 분명한 작은 기능부터 실행해 봅니다.

왜 그런지 묻고

파일과 코드가 어떤 역할을 하는지 설명을 받습니다.

좋은 결과와 비교합니다

읽기 쉬운 화면과 정확한 처리의 차이를 구체적으로 찾아봅니다.

만들기 시작하는 문턱은 낮아졌습니다. 잘 만드는 지식과 안목은 계속 쌓아갑니다.

어떤 부분이 궁금하신가요?

Q&A · 5분



처음 시작하는 방법

화면과 처리의 역할

내 업무에서 만들 수 있는 작은 기능

오늘 기억할 것은 요청, 실행, 확인, 수정의 흐름입니다.
무엇을 요구하고 어떻게 판단할지는 지식과 경험이 채워줍니다.

누구나 시작할 수 있습니다. 더 잘 만드는 힘은 지식과 안목에서 나옵니다.

용어 자세히 보기

Vibe Coding

원하는 기능을 자연어로 설명하고, AI가 만든 코드를 실행하고 수정하며 프로그램을 만드는 접근입니다.

이 교육에서는 요청, 실행, 결과 확인, 수정의 흐름으로 경험합니다. AI가 코드를 작성하더라도 목적과 업무 기준을 설명하고 결과를 판단하는 일이 필요합니다.

시작하는 문턱은 낮아지지만, 복잡한 기능을 잘 만들려면 프로그램 지식과 화면을 보는 안목을 계속 쌓아야 합니다.

브라우저

웹 페이지를 열고 사용하는 프로그램입니다.

Edge, Chrome 등이 해당합니다. HTML과 CSS로 화면을 표시하고 JavaScript를 실행합니다. 브라우저에서 할 수 있는 일과 서버에서 하는 일은 구분해서 봅니다.

오늘의 HTML 파일은 브라우저에서 직접 열어 사용할 수 있고, 외부 서비스 연결 없이 준비된 체험이 동작합니다.

서버

다른 프로그램의 요청을 받아 기능이나 데이터를 제공하는 프로그램 또는 그 실행 환경입니다.

브라우저가 요청하는 계산, 저장, 인증 같은 일을 맡을 수 있습니다. 서버 쪽 처리 부분을 백엔드라고 부릅니다. 모든 작은 도구에 별도 서버가 필요한 것은 아닙니다.

여러 사람이 같은 자료를 조회하거나, 비밀 키를 사용해 AI 서비스에 요청할 때 서버 쪽 처리를 둘 수 있습니다.

Node.js

JavaScript를 브라우저 밖에서 실행하는 환경입니다.

서버 프로그램이나 일부 개발도구가 이를 사용합니다. JavaScript를 사용하는 모든 화면에 별도로 설치해야 하는 것은 아닙니다.

오늘처럼 브라우저에서 여는 단일 HTML 파일에는 Node.js 설치가 필요하지 않습니다. 만들 프로그램에 맞춰 필요한지 확인합니다.

패키지

프로그램 기능이나 도구를 설치하고 배포할 수 있게 묶은 단위입니다.

라이브러리나 실행 프로그램이 담길 수 있습니다. 패키지 관리자는 이런 묶음을 설치하고 버전을 관리하는 도구입니다.

Python의 pip, JavaScript 생태계의 npm 등이 있습니다. 특정 패키지가 왜 필요한지, 어떻게 설치하는지는 에이전트에게 물어볼 수 있습니다.

Markdown

제목과 목록 같은 문서 구조를 간단한 문자 기호로 표현하는 형식입니다.

파일 확장자는 보통 .md입니다. 코드를 실행하는 프로그램이 아니라 사람이 읽을 설명과 규칙을 적는 데 자주 사용합니다.

README.md에는 실행 안내를, AGENTS.md에는 지원하는 에이전트가 참고할 작업 기준을 기록할 수 있습니다.

코딩 에이전트

파일과 도구를 사용해 코딩 작업을 수행하는 AI 시스템입니다.

일반적인 코드 답변을 넘어, 연결된 작업 공간에서 파일을 읽고 수정하고 명령을 실행할 수 있습니다. 가능한 행동은 제품, 연결된 도구, 권한에 따라 달라집니다.

게임의 점수 표시를 수정하고 실행 결과를 확인하도록 요청할 수 있습니다. 완료 메시지와 실제 결과는 함께 확인합니다.

GUI

Graphical User Interface. 화면의 버튼, 입력란, 목록 등으로 조작하는 사용자 인터페이스입니다.

정보가 어떤 순서로 보이는지, 글자가 읽히는지, 다음 행동이 분명한지를 함께 봅니다. 보기 좋은 장식만 뜻하지 않습니다.

“예쁘게”보다 “검토 건수를 먼저 보여주고 주요 버튼의 이름을 구체적으로 바꿔줘”라고 요청할 수 있습니다.

프로젝트 폴더

하나의 프로그램에 필요한 파일과 자료를 모아 둔 폴더입니다.

코드, 참고 자료, 설정, 실행 안내를 함께 관리합니다. 에이전트가 어느 폴더에서 작업하는지 확인해야 다른 작업과 섞이지 않습니다.

star-game 폴더에 index.html과 실행 안내를 두고, 그 폴더에서 변경을 이어갑니다.

터미널

문자 명령을 입력하고 프로그램의 출력을 보는 창입니다.

PowerShell 같은 셸이 명령을 해석합니다. CLI는 버튼 대신 명령어로 도구를 사용하는 방식을 뜻합니다. 터미널, 셸, CLI는 연결된 개념이지만 같은 뜻은 아닙니다.

python app.py에서 python은 실행할 도구, app.py는 실행할 파일입니다. 파일 위치와 설치 상태가 맞아야 실행됩니다.

Python

데이터 처리나 서버 프로그램 등에 사용하는 프로그래밍 언어입니다.

Python 코드를 실행하는 환경을 설치해야 해당 코드를 동작시킬 수 있습니다. 웹 화면을 구성하는 HTML과 역할이 다릅니다.

점수를 읽어 계산하는 app.py를 Python으로 실행할 수 있습니다. 모든 작은 HTML 도구에 Python이 필요한 것은 아닙니다.

HTML

웹 화면의 구조와 내용에 의미를 부여하는 마크업 언어입니다.

제목, 문단, 입력란, 버튼 같은 요소를 표현합니다. 그 자체가 AI 기능이거나 서버를 뜻하지는 않습니다.

버튼이 있다는 구조는 HTML로, 버튼의 색과 간격은 CSS로, 누른 뒤의 변화는 JavaScript로 구성할 수 있습니다.

CSS

화면의 색, 크기, 간격, 배치 같은 표현을 정하는 스타일 언어입니다.

같은 HTML이어도 CSS에 따라 읽는 순서와 사용 경험이 달라집니다. 화면 크기에 맞는 배치도 다룹니다.

점수를 크게 표시하고 주된 버튼을 눈에 띄게 배치합니다. 색을 바꾼다고 잘못된 계산이 고쳐지는 것은 아닙니다.

JavaScript

웹 화면의 입력과 상태 변화 등을 구현하는 프로그래밍 언어입니다.

브라우저에서 버튼 클릭에 반응하고 데이터를 계산할 수 있습니다. 브라우저 밖의 실행 환경에서도 사용할 수 있습니다.

오늘의 순위 도구는 브라우저 안에서 입력을 읽고 정렬합니다. 외부 AI나 별도 서버를 호출하지 않습니다.

프론트엔드

사용자가 직접 보고 조작하는 부분입니다.

웹 프로그램에서는 브라우저에 표시되는 화면과 그 동작이 대표적입니다. 화면 구성뿐 아니라 입력 검사나 일부 계산도 할 수 있습니다.

이름과 점수를 입력하는 칸, 정렬 결과, 오류 안내가 해당합니다. 모든 처리 로직이 반드시 백엔드에 있어야 하는 것은 아닙니다.

백엔드

서버 쪽에서 요청을 받아 처리하는 부분입니다.

데이터 확인, 공통 업무 규칙, 권한 검사, 저장, 다른 서비스 호출 등을 맡을 수 있습니다. 브라우저만으로 충분한 도구에는 별도 백엔드가 없을 수도 있습니다.

여러 사용자가 공유하는 자료를 저장하거나, 서버에서 인증 정보를 사용해 AI 서비스에 요청할 때 필요할 수 있습니다.

데이터베이스

데이터를 체계적으로 저장하고 조회하는 저장소입니다.

화면을 닫은 뒤에도 자료를 보관하거나 여러 조건으로 찾아 쓰는 데 활용합니다. 어떤 자료를 어떻게 보관할지에 따라 구조가 달라집니다.

현재 순위 도구는 입력을 메모리에서만 처리합니다. 계속 보관하려면 파일 저장이나 데이터베이스 등 저장 방법을 별도로 설계해야 합니다.

API

Application Programming Interface. 프로그램끼리 정해진 방법으로 기능을 요청하고 결과를 받는 접점입니다.

웹 API에서는 주소, 요청 형식, 인증 방식과 응답 형식 등을 약속합니다. API는 AI에만 있는 것이 아닙니다.

브라우저가 서버에 점수 계산을 요청하거나, 백엔드가 AI 서비스에 문장 분류를 요청할 수 있습니다. 실패 응답도 처리해야 합니다.

AGENTS.md

이를 지원하는 코딩 에이전트에 프로젝트의 작업 규칙을 전달하는 문서입니다.

Markdown 형식의 텍스트 파일입니다. 읽는 위치와 적용 범위는 사용하는 도구에 따라 다르며, 파일이 있다는 것만으로 모든 도구가 자동 적용하지는 않습니다.

“한국어 화면으로 작성”, “수정 후 정상·누락·경계 입력 확인” 같은 기준을 남깁니다. 규칙을 적어도 결과 확인은 필요합니다.

Git

파일의 변경 이력을 기록하고 비교하는 버전 관리 도구입니다.

확인한 변경을 커밋으로 기록하고 과거 내용과 차이를 볼 수 있습니다. 자동으로 모든 수정이 기록되는 것은 아닙니다. GitHub는 저장소를 온라인으로 보관하고 협업하는 서비스입니다.

화면 수정 전후를 비교하거나, 잘 동작하던 상태를 찾아 복구 방법을 검토할 수 있습니다.

API 키

서비스를 사용하는 주체나 권한을 확인하는 인증 정보입니다.

노출되면 다른 사람이 권한이나 사용량을 악용할 수 있습니다. 브라우저에 전달한 코드와 값은 사용자에게 보일 수 있으므로, 비밀 키는 서버 측에서 관리합니다.

발표자료·공유 파일·브라우저 화면에 실제 키를 넣지 않습니다. 이 자료에는 실제 키 입력란이나 서비스 연결이 없습니다.

환경변수

실행 환경에서 프로그램으로 전달하는 설정값입니다.

코드 자체를 바꾸지 않고 주소나 실행 옵션을 다르게 주는 데 사용합니다. 비밀값도 전달할 수 있지만, 자동 암호화나 유출 방지를 보장하지는 않습니다.

서버 실행 시 서비스 주소와 인증 정보를 설정합니다. 로그에 출력하거나 화면에 전달하면 노출될 수 있습니다.

라이브러리

다른 프로그램에서 가져다 쓸 수 있도록 만든 기능 모음입니다.

패키지는 설치·배포 가능한 묶음이고 라이브러리를 포함할 수 있습니다. 패키지 관리자는 이런 묶음의 설치와 버전을 관리합니다.

표 읽기, 차트 그리기 같은 기능을 가져다 쓸 수 있습니다. 무엇이 필요한지는 만드는 프로그램에 따라 달라집니다.

가상환경

Python 프로젝트마다 패키지 설치 환경을 분리하는 공간입니다.

서로 다른 프로젝트에서 필요한 패키지 버전이 충돌하는 일을 줄입니다. 가상 머신이나 별도의 컴퓨터를 만드는 것은 아닙니다.

프로젝트 A와 B가 서로 다른 버전의 같은 라이브러리를 쓸 때 각 환경을 따로 준비할 수 있습니다.
